

Autonomous Racing: A Comparison of SLAM Algorithms for Large Scale Outdoor Environments

Felix Nobis

nobis@ftm.mw.tum.de

Chair of Automotive Technology
Technical University of Munich

Leonhard Hermansdorfer

Chair of Automotive Technology
Technical University of Munich

Johannes Betz

Chair of Automotive Technology
Technical University of Munich

Markus Lienkamp

Chair of Automotive Technology
Technical University of Munich

ABSTRACT

The task of simultaneous localization and mapping (SLAM) is a widely studied field in robotics research in the last decades. The goal of SLAM is to create an accurate map of the environment considering uncertainties in the pose as well as the environmental perception of the robot. Historically SLAM algorithms are applied in the field of indoor robotics. Recent developments in the area of autonomous driving surge a focus for SLAM applications in large scale outdoor environments.

Two notable open source SLAM software packages are Gmapping and Google Cartographer. This paper focuses on a qualitative comparison of the aforementioned algorithms for such a scenario. We discuss the underlying algorithmic differences of the two packages. This serves as the foundation to present the SLAM results for different parameter configurations. We evaluate the accuracy of the resulting maps and the respective computational limitations. The maps are further evaluated against manually measured ground truth track boundaries. We show that the existing approaches can be adapted to large-scale outdoor environments.

CCS CONCEPTS

• **Applied computing** → **Engineering**; • **Mathematics of computing** → *Probabilistic algorithms*.

KEYWORDS

Autonomous driving; perception; LIDAR; Gmapping; Cartographer; large scale SLAM; sparse feature SLAM

1. INTRODUCTION

In the process of developing a fully autonomous vehicle the perception of the environment is one of the three core research areas. The other ones being path-planning and control [22]. In some applications it may be feasible to only perceive the current local

environment of the vehicle. Planning the path in the currently detected lane while avoiding perceived obstacles. In a global context, where the goal is to drive from A to B, a global map is needed to ensure to reach the target position. The need for a global map is of definite importance in the area of autonomous racing. The vehicle needs to know the curvature of the track ahead, before the sensors are physically able to measure it e.g. due to obstruction of barriers. Only with this knowledge, it is able to plan an optimal racing trajectory. Without this information the vehicle may enter the curve at a too high or too low speed.

The work for this paper is part of the effort of TU Munich to participate in a racing series for autonomous cars. The results of this paper were tested in an collaborative event of the Roborace series and TUM in May 2018 in Berlin [6]. The aim of the Roborace project is to develop algorithms for the appliance to autonomous vehicles, both on the race track and on public roads. An overview of the scope at our university can be found in [3].



Figure 1: Roborace car on a Formula E track

This paper develops a strategy to create grid maps for roads which are bounded by physical walls. Optical features, e.g. lane markings are not incorporated into these grid maps. The Formula E race tracks to which we apply this strategy are bounded by concrete walls, while the lane markings on the ground do not always represent the course of the temporary race track.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

International Conference on Virtual and Augmented Reality Simulations, ICVARS
© Association for Computing Machinery.

DOI: 10.1145/3332305.3332319

For the environmental perception of autonomous vehicles a great variety of sensor technology is available. Widely used sensors include cameras, radar and LIDAR. The use of LIDAR sensors is favorable in the given scenario as they enable the direct measurement of the distance to the concrete walls which bound the track. The race track boundaries are constructed perpendicular to the road surface as shown in Figure 1. In theory, it is possible to construct a map with only the distance information of a 2D LIDAR scan by estimating the vehicles pose through scan matching. However, the environment of the race track possesses only sparse features with several long straight corridors. This implies a challenge for the scan matching in the localization step. During this absence of distinguishable features, the need for a good pose prior or odometry prior is crucial to enable the SLAM process.

In the remainder of this paper we briefly cover related research in Section 2.. We present the hardware setup available to us in Section 3.. In Section 4. we review the major concepts and differences between the used SLAM packages and present their adaption to our use case. Then we present the results in Section 5. and close the paper with our conclusions.

2. RELATED WORK

A good fundamental overview of the main concepts and ideas in the field of robotic mapping and SLAM can be found in [25]. In literature, many SLAM applications make use of camera images or LIDAR range measurements in combination with an odometry prior [15], [9], [11]. On the other hand algorithms which solely rely on the information of the environmental sensor, without using any further odometry or pose measurements are also available [7], [13]. An exhaustive overview of SLAM methods and their application for autonomous driving is given in [5].

Most developments and application of SLAM are optimized for indoor environments with small scale robots. This includes the widely used open source Gmapping and Hector SLAM packages [13], [9]. A major design difference between the approaches lies in the incorporation of odometry and pose information in the SLAM approach of Gmapping. This makes this package generally applicable to environments where scan matching cannot succeed or only succeeds to a certain level. Gmapping relies on a particle filter to estimate the current pose of the vehicle. A broad overview of the workings of particle filters can be found in [10]. A parameter study for the Gmapping approach has been conducted in [1].

In contrast, Hector SLAM only estimates the vehicle pose from the scan match result. This leads to insufficient accuracy in feature sparse environments. Furthermore, Hector SLAM does not provide a global loop closure, a continuous drift will therefore not be corrected when the same location is traversed for a second time.

Through the advancement of LIDAR and camera technology for autonomous vehicles, the development of SLAM also makes a shift towards application to large scale environments. In the field of SLAM with camera ORB SLAM is a notable approach for large scale environments [17].

Just like Gmapping, the Google Cartographer is a SLAM approach using LIDAR as an input source [11]. The core concepts of this approach will be surveyed in the course of this paper. An

advantage to the large scale application is the efficient use of computational resources and the resulting real-time applicability.

The application of these algorithms is mostly still evaluated in small scale indoor environments [18]. As an exception [4] applied SLAM to generate a map of a large scale urban environment. Santos et al. [24] compared different SLAM implementations for ROS. Kamarudin et al. [12] presented an evaluation of the Gmapping and the Hector package for an indoor SLAM application. Our paper aims to present an evaluation for Gmapping and the Google Cartographer in the application to large scale outdoor SLAM.

3. HARDWARE

The vehicle platform used for this research is the so-called DevBot of Roborace. The vehicle is equipped with a variety of sensors, to enable it to race against other course in an autonomous competition. In this section we only mention those sensors relevant for this research. A more detailed insight to the vehicle is given in [3].

The environmental perception is realized through four Ibeo ScaLa B2 LIDAR sensors [2]. Two are positioned on the front wing of the vehicle, one on either side. Two more are located at either side of the cockpit, pointing outwards-backwards. They enable a close to 360° field of view around the vehicle. This setup is shown in Figure 2. Each of the LIDARs possesses four layers to measure the environment in 3D. The vertical angle between each layer is 0.8°, resulting in a limited information gain in the z-coordinate. As we are detecting vertical walls, the points can therefore be projected to an 2D x-y plane for the map generation without relevant loss of information. This is necessary for the Gmapping approach, since it can only process 2D laser scans.

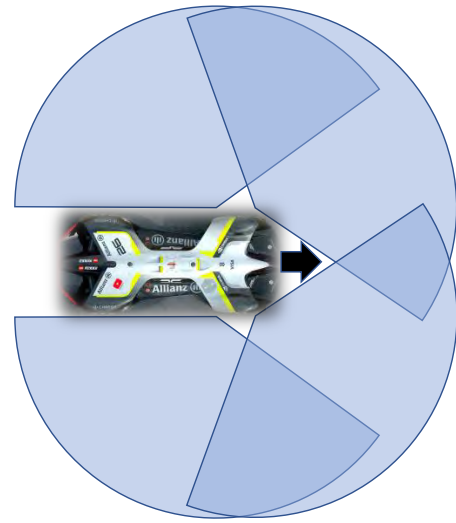


Figure 2: Lidar configuration on the vehicle

The odometry and pose prior for the SLAM algorithms is provided through an Oxts RT4000 system [21]. This system incorporates a global navigation satellite system (GNSS) receiver as well as an Inertial Measurement Unit (IMU). It enables the measurement of an accurate odometry and pose prior to enable the SLAM process in an environment with only a small number of distinctive features.

The sensor readings of the four LIDARs are fused into a single point cloud and transformed into the center of gravity of the vehicle. The IMU is physically located in the center of gravity making further transformations obsolete.

The data is transmitted over Ethernet to the NVIDIA Drive PX2 ECU [19]. On the ECU the data is introduced into the Robot Operating System (ROS) framework as an interface to the SLAM algorithms [23]. In localization mode, the point cloud data is evaluated on the ECU. For the SLAM process, we save the data in the format of a ROS bag to process it offline. This is necessary for the Gmapping approach since in this case computational resources are not sufficient for real time SLAM. The SLAM process is run offline, on a laptop running Ubuntu 16.04 on an Intel Core i7-6820HQ at 2.7 GHz with 16 GB of RAM. After the maps are calculated on this external computer, they are used to plan a racing trajectory and to perform online LIDAR localization in autonomous mode.

The hardware setup of four individual LIDARs requires a sensor fusion of the individual measurements into a global point cloud to further process the data. To achieve this, the exact positions and orientations of the sensors with respect to the centre of gravity and a vertical axis need to be known. In practice, these values are hard to determine accurately. Every time the vehicle has to be cleaned or revised, the sensors need to be de-mounted and remounted again. The sensors are fixed with screws in a rack. The position can slightly vary after each new mount and the yaw orientation could be chosen arbitrarily. Adjusting all four angular orientations to a specified value is therefore not feasible. This leads to small angular offsets in the sensor readings.

The raw accumulation of the LIDAR scans into a global coordinate frame of consecutive sensor readings is shown in Figure 3. The noise shows especially in the depicted corner and in form of measurements of the ground and overhead structures. In the following, we show that the SLAM packages can successfully deal with this type of systematic noise and create sufficiently accurate grid maps. The same section of the race track will be revisited in Section 5. for a comparison of the resulting maps of both SLAM algorithms.

4. SLAM IMPLEMENTATIONS

In this section we compare two SLAM implementations that are the widely used in robotics research towards their applicability in autonomous driving on a race track. We describe the general concepts of the algorithms and present the necessary adaptations for our use case.

4.1 Gmapping Concepts

The SLAM algorithm underlying the Gmapping package [9] is based on a Rao-Blackwellized particle filter. The algorithm further uses an efficient proposal distribution for each particle which considers scan-alignment in addition to odometry and pose measurements. Each particle of the particle filter represents its own grid map and the underlying vehicle trajectory. The particle filter estimates the probability distribution $p(x_{1:t}, m | z_{1:t}, u_{1:t-1})$ for the vehicle trajectory $x_{1:t}$ and the map m considering the measurements of the laser scanner $z_{1:t}$ and the odometry measurements $u_{1:t-1}$. To facilitate the estimation of this distribution the Rao-Blackwellization technique is applied. In this way the probability distribution can be

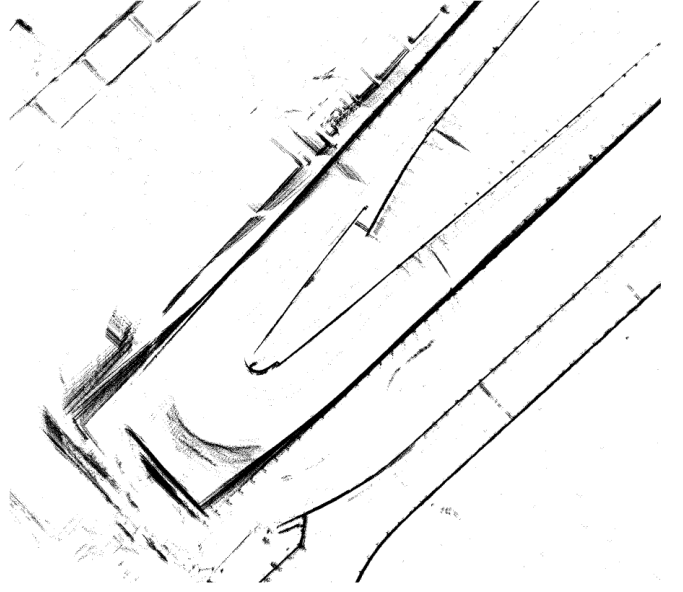


Figure 3: LIDAR measurements in global coordinate frame

factorized in the following way:

$$p(x_{1:t}, m | z_{1:t}, u_{1:t-1}) = p(m | x_{1:t}, z_{1:t}) p(x_{1:t} | z_{1:t}, u_{1:t-1}). \quad (1)$$

The first part of this equation can be computed analytically. Only the second factor needs to be estimated by the particle filter.

Generally particle filters deal with the problem of particle depletion. This means that during the particle re-sampling step of the filter the correct particle might be eliminated. The authors of the Gmapping algorithm therefore use an adapted estimation of the current state x_t . This estimation does not only depend on the past state x_{t-1} and odometry measurement u_{t-1} but furthermore uses the information of the current scan measurement z_t matched against the existing map m_{t-1} . This leads to a more accurate prior in comparison to a solely odometry based estimate, so that the risk of eliminating a particle close to the correct solution is reduced.

Gmapping uses the scan matching algorithm developed for the CARMEN toolkit [14]. The scan matching process in Gmapping enables global loop closure, once the vehicle reaches the same portion of a track for the second time to correct local drift. The standard API of Gmapping provides over 30 parameters to adapt the behavior of the scan matcher, the particle filter and general ROS settings. In the following we review some important parameters and their effect for our use case.

4.2 Gmapping Parameters

Table 1 presents the parameters we adapt to map different race track environments. We present their effect in the following paragraphs.

The map resolution is reduced to 20 cm. This is necessary to reduce the computational load of the algorithm. This coarse grid value is still acceptable for our approach. Further measures will be applied to reduce the computational load.

Table 1: Gmapping Tuning parameters

Parameter Name	Default	Adapted
Map resolution	5 cm	20 cm
Number of Particles	30	5
Scan Matcher Minimum Score	0	500
Standard deviation ρ with respect to ρ	0.1	0.0
Standard deviation ρ with respect to θ	0.2	0.0
Standard deviation θ with respect to ρ	0.1	0.0
Standard deviation θ with respect to θ	0.2	0.0

The number of particles to estimate the pose of the vehicle has to be reduced to 5. This is also motivated due to computational limits in conjunction with operating at a speed of about 40 km/h for track mapping, which is significantly higher than in common indoor scenarios. The large scale environment of roughly 600 m · 600 m and the high number of laser beams to be processed for each of the four LIDARs add to the computational burden of the studied scenario. The vehicle moves through the track at around. For each particle, Gmapping saves the trajectory of the vehicle and a corresponding map, of which the most probable particle will be used for the map estimation. This explains that every additional particle is costly in the matching and resampling step, as the computational need scales linearly with each new particle. In the case of an unreliable odometry estimation e.g. from wheel speed sensors, a higher number of particles is advised to enable a wide enough spread of the estimations to find the true current pose. Our intent is to rely strongly on the fused odometry and pose measurements by the IMU and GNSS sensors so that we can operate with a smaller number of particles.

The Scan Matcher Minimum Score determines a threshold confidence value for the map/scan alignment of the scan matcher. In feature-rich environments, the iterative scan matcher is able to find the correct solution to align the scan to the real orientation of the map online. In this case the laser-estimated pose of the vehicle is processed for the localization step of the SLAM. In the environment we are operating in, the fused odometry and pose measurement is highly accurate. The features for scan matching however are relatively sparse in the race track environments. In order to further rely on the odometry and pose measurements, we augment the Scan Matcher Minimum Score to 500. If this score cannot be reached e.g. due to a lack of features, the LIDAR pose estimation is not used for localization.

The last four parameters of the table represent standard deviations in the Gaussian motion model which is used to model uncertainties in the odometry measurements. The vehicle is modeled as a simple point in the centre of gravity in a 2D-plane with three degrees of freedom. We decide to set the standard deviations for the Gaussian distributions to zero for all components to directly use the odometry and pose measurements as the input source for the scan matching.

All of the steps above lead to a heavy confidence of the algorithm in the odometry source. In cases where the scan matcher is not able to find a solution quantified higher than the minimum score, the odometry and pose measurement is the only source for the localization step of the SLAM algorithms. As there is no further adaption

of the pose, it is possible to use Gmapping as a tool for mapping with known poses [16]. We will show that the map estimation is highly accurate for the given environment in Section 5.

In our research, we apply the Gmapping approach to large-scale outdoor environments with a good prior pose estimation, as well as to indoor applications with a less accurate source of odometry and pose measurements in feature rich environments. In the indoor case, the scan matcher augments the pose estimation through scan alignment significantly. This makes Gmapping a valuable tool in the field of SLAM despite that the original developers no longer continue their work on the tool.

4.3 Google Cartographer Concept

The second SLAM tool we review in this paper is the Google Cartographer [11] [8]. This tool is still actively maintained by the developers. Like Gmapping, the Cartographer can be run as a real-time SLAM tool. Additionally, it comes with an adapted implementation to process recorded data. In this implementation, it comes with the advantage of being able to process the data faster than real-time. It runs at the maximum speed the current hardware configuration enables.

In difference to Gmapping, the Cartographer is able to create 3D maps of the environment. Furthermore, it can deal with a greater variety of data input. This includes 2D laser scans as in Gmapping. Furthermore, it can deal with point clouds represented by their x , y and z coordinates directly. It is further possible to input several sources of lasers or point clouds into the algorithm.

The Cartographer can use pose and velocity measurements for the movement estimation to the algorithm. Furthermore, it can use readings from an IMU. In the 2D case, these measurements are used to stabilize the scan aligned to the horizontal plane.

In the 3D SLAM case, the availability of an IMU is necessary for the algorithm to function. In 3D mode, the Cartographer can also process GPS signals directly to stabilize the drift of the pose estimation. It needs a detailed enough point cloud to be able to apply the scan matching step in the 3D case. With the sensors available for this research, the vertical resolution is too sparse to successfully create a 3D map.

In the end, we want to create a ROS compatible 2D map for our path-planning algorithm, so that the limitation to 2D is not severe. Nonetheless, it remains interesting to study if the SLAM result from 3D would lead to an even more precise map when it is projected back to the 2D plane.

The SLAM process in the Cartographer is based on a local scan matching approach combined with a global graph based SLAM for matching local sub maps against the current laser scan. Similar to Gmapping the Cartographer iteratively solves an optimization problem to align the current scan to an existing map. The difference is that Cartographer only matches against a local map. The obtained transformation is used to correct the pose estimate of the odometry and pose measurement. Depending on how reliable the odometry and pose information is, this alignment is done with two different algorithms. For an unreliable source of odometry, a brute force approach can be used before the precise alignment is found by solving a nonlinear least squares optimization problem. For a

reliable odometry and pose source only the nonlinear optimization step is computed.

Only a certain amount of scans is matched into a local sub map using this principle. The number of scans fused into each sub map can be specified by a parameter.

Once a sub map is built, it can no longer be changed. In the case of a global loop closure, a new sub map will be generated overlaying the old sub map without changing its properties. This way the Cartographer achieves to be computationally efficient.

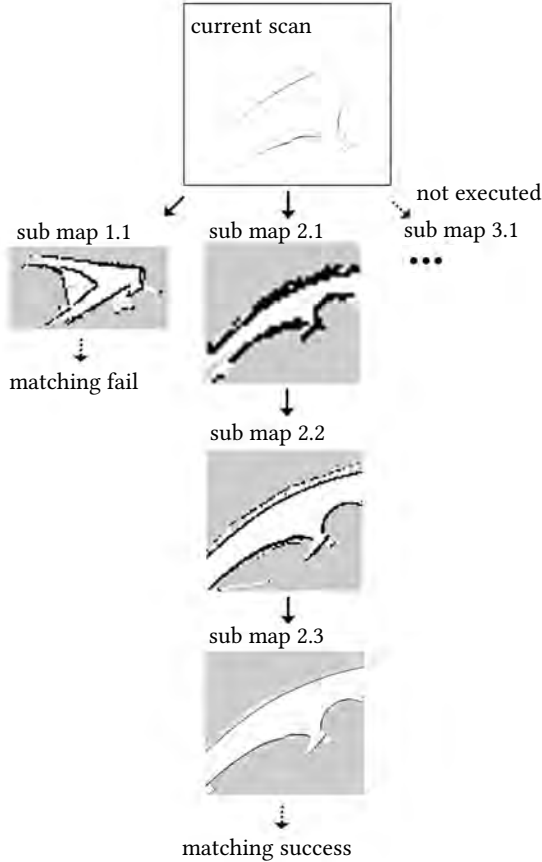


Figure 4: Tree structure for the loop closure process

For global loop closure, the local sub maps get aligned against the current scan in a second optimization problem. A multi-resolution approach is used in this step [20]. The local sub maps are stored in different resolutions in a tree structure. This tree structure is visualized in Figure 4. The tree branches are indicated by solid arrows. The current scan is matched against a low resolution representation of a sub map in the first branch of the tree. The search in a branch is aborted if the scan matcher does not reach a certain threshold at any resolution level. This happens for the sub map 1.1 in the example. The search is then reinitialized for the next branch of the tree which represents another sub map. Should the solution of the optimization reach the threshold, the scan is matched against a higher resolution version of this sub map. This is repeated until

the scan is matched to the sub map in the original resolution and a loop closure is detected. In the example the current scan matches with the sub map 2. Further sub maps are not checked in this case.

If the optimizer does not find a feasible solution for any of the branches, no loop closure is computed. The tree search approach makes the matching process efficient as the optimization at low resolutions is less costly and the branch is left once the threshold is not reached.

4.4 Google Cartographer Parameters

Table 2 presents the parameters we adapt to map different race track environments. We present their effect in the following paragraphs.

Table 2: Cartographer Tuning parameters

Parameter Name	Default	Adapted
Map resolution	5 cm	15 cm
Submap Scan Number	35	70
Local Slam		
Occupied Space Weight	1	1
Translation Weight	10	1e10
Rotation Weight	40	1e10
Global Slam		
Local Slam Translation Weight	1e5	0
Local Slam Rotation Weight	1e5	0
Odometry Translation Weight	1e5	1e10
Odometry Rotation Weight	1e5	1e10

The map resolution is adapted to the scale of our environment. The computational resources are sufficient to create a map at a resolution of 5 cm. However we choose the resolution to 15 cm for the use in our vehicle. This is because of two practical reasons: First, the resulting grid map size in the memory scales quadratically with the resolution. The uncompressed grid maps are processed in autonomous driving mode for LIDAR localization. The high resolution grid maps lead to stability problems in the form of crashes of the ROS map server package. Secondly, the used LIDAR scanner is able to scan the environment at a resolution of 10 cm, so that a finer resolution for the grid map can not be created with this measurement source.

The number of scans which are fused into a single sub map is doubled for our application. More accumulated scans in a single sub map lead to more connected features in either sub map. Our experiments have shown that the global drift of the resulting map can be reduced in many cases by incorporating larger sub maps.

The parameters of the Cartographer follow a similar structure to the Gmapping approach. The first group of parameters is used to set the behavior of the local SLAM. The Occupied Space Weight parameter determines how much we trust the laser scan matching for the pose estimation. The scan is modeled as the endpoints of the laser in the optimization problem. The Translation Weight and Rotation Weight specify how much we trust the prior of the vehicles pose coming from the external odometry and pose source. Note that the units of these values are not scaled equally. Higher values however, still represent a higher trust in the corresponding measurement. In this way we choose to heavily rely on the odometry and pose prior.

In the global SLAM case the tuning parameters weight between the pose estimation of the local SLAM and the pose estimation of the odometry measurements. We choose the parameters in a way that the fused odometry and pose measurement is strongly trusted.

In comparison to the Gmapping results which can be regarded as mapping with known poses for sections where no scan matching is achieved, the Cartographer approach uses the scan matching to adapt its pose, thus performing a full SLAM in the feature sparse environment. We applied further changes to the Cartographer in other work, in order to make it work as an algorithm for mapping with known poses as well.

5. RESULTS

In this section we evaluate the 2D grid maps resulting from the mentioned SLAM tools and configurations with data sets collected during a manual driving session on the Formula E race track in Berlin 2018. First, we compare the maps of both algorithms with each other. In the second step, we compare the maps against externally measured ground truth boundaries. One should keep in mind, however, that even the ground truth boundaries could not be measured down to centimetre accuracy with the available means. Additionally, we compare the computational resources used during the mapping process of the respective tools.

The map needs to be created in relation to a fixed global coordinate system with a defined origin to incorporate the GNSS measurements into the localization in autonomous driving mode. The map representation of the Cartographer uses the first data point it receives as the origin of its environment. This may be a reasonable approach for many other use cases. For us, it is not desired since it changes the global position and orientation of the aligned map. In order to utilize the Cartographer map for our global localization approach based on a fused LIDAR and GNSS pose estimate, we transform the Cartographer map back into fixed global coordinates after the SLAM process is finished.

Figure 5 compares the mapping result of a long straight and a hairpin curve of the Berlin Formula E race track for both algorithms. The detected walls of the Cartographer algorithm are shown in red. The walls detected by Gmapping are shown in blue. One can see that the walls of both maps coincided for the major part of the track. In comparison to the raw point cloud data in Figure 3, one can see that either algorithms is able to reduce the noise of the sensor input to detect firm walls in the environment. Furthermore, the LIDAR scanner detected parts of the race track in the hairpin curve, as the roll angle of the vehicle led to the LIDARs pointing towards the ground. The SLAM algorithms did correctly classify this area as free space. The same holds true for overhead detections and detections of the track during braking.

Figure 6 shows a significant SLAM error we encountered using the Cartographer algorithm on the Berlin race track. The lateral offset of the right hand side wall is up to 1.35 m. In the section preceding this part of the track, the scan matcher found a solution which does not coincide with the real surroundings. The preceding curve is distorted slightly which results in the pictured straight to inherit a lateral offset. The Gmapping map, again in blue, detects the walls at the correct position in space. For different parameter configurations and numbers of loop closures, we were able to reduce

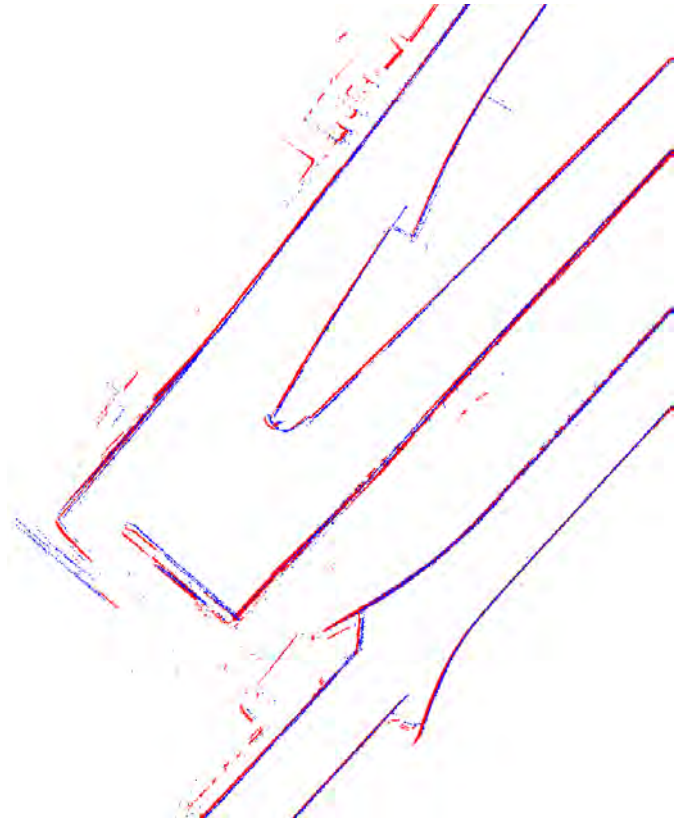


Figure 5: Good map alignment for Google Cartographer (red) and Gmapping (blue) algorithms

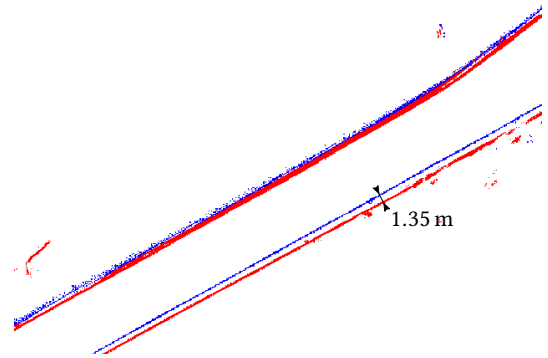


Figure 6: Map alignment error for Google Cartographer (red) and Gmapping (blue) algorithms

the error in the Cartographer map. However in our work, we were not able to prevent any misalignment on the global map from occurring altogether. Additionally to the parameters mentioned earlier, further scan matching thresholds together with a plethora of solver and map parameters were adapted to find a feasible solution. Due to their vast number, a parameter grid based tuning of the parameters is not realizable. We therefore approached the problem to our best knowledge by tuning the most influential parameters.

However, we can not rule out that a parameter combination exists which could overcome the matching errors.

In order to globally compare the mapping results, a ground truth in the form of measured track boundaries with an external GPS device is used. The GPS boundaries are measured with an offset of 1.0 m to 1.5 m to the walls. The Gmapping map coincides well with the ground truth for the entire race track. The distance between the walls and the ground truth is nearly constant which can be seen in Figure 7. The ground truth is shown in green. The detected walls in black. Unknown areas are marked grey.

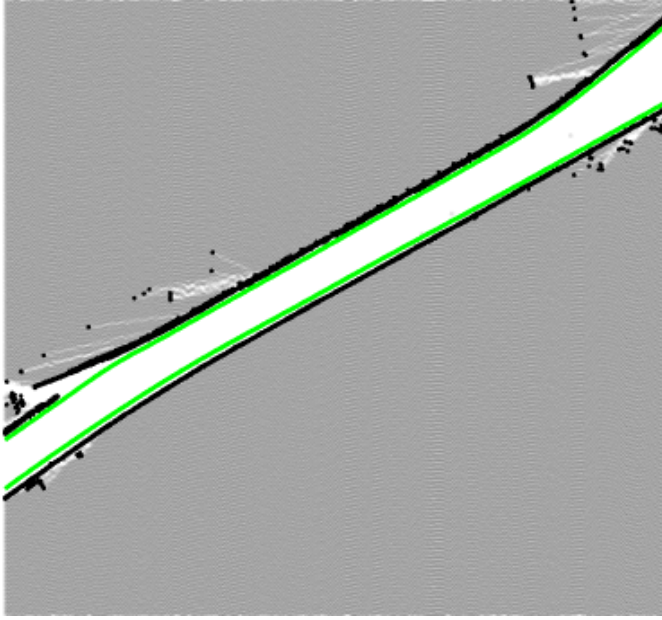


Figure 7: Correct alignment of the Gmapping map. Ground truth in green.

The same section of the track is shown for the Cartographer map in Figure 8. The mapping error is visible. The detection of the lower right wall lies too far outside of the ground truth boundary. The left wall is partially mapped coinciding with the ground truth boundary. The ground truth was measured inside the track with an offset so we can reason that the map is erroneously translated. Additionally to the competition race track several further data sets were used to tune the mapping process. In some of those environments, the misalignment between Cartographer and Gmapping SLAM did not reach a significant value. For those tracks no ground truth information is available. Due to the conformity of the maps of the two algorithms however, we deduce that the Cartographer achieved more favorable results in these cases. The application to the large scale feature sparse racing environment can be achievable. Still, before using the resulting map for the localization task, the validation of the map through a comparison to another source is favorable. This reduces the risk of mislocalization.

In addition to the raw output of the algorithms we evaluate the computational limitations of either algorithm. The Cartographer is able to process the data faster than real-time to create a grid map of the environment. It makes use of the different cores of

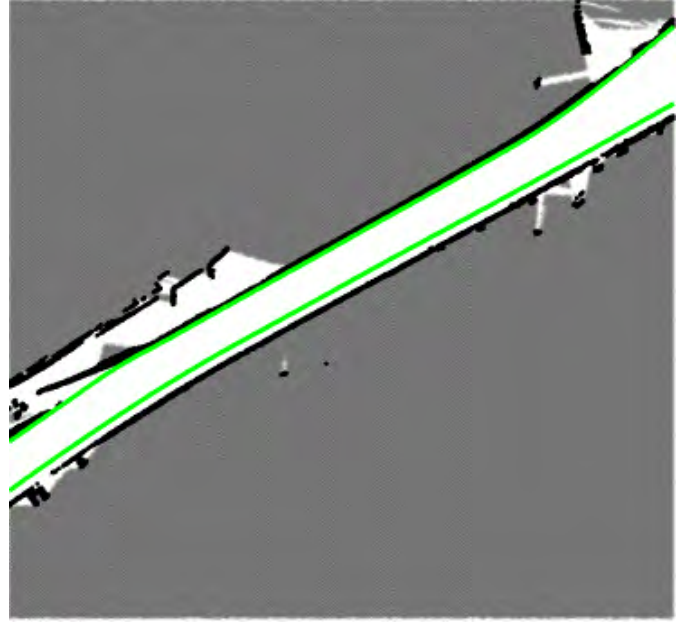


Figure 8: Translational error of the Cartographer map. Ground truth in green.

the computer to distribute its calculations. During the real time mapping the Cartographer uses 30 % of the CPU power available on the computer described at the end of Section 3.. It is possible to create a map even faster with the mentioned offline node.

In contrast, we were not able to run the Gmapping approach in real-time for our environment. To receive a suitable map the data was processed at 1/4 playback speed in Gmapping. Even in this configuration the CPU operated at the limit of its capacities.

6. CONCLUSIONS

In vast parts of the track, both tools calculate a reasonable representation of the race track environment. This is a favorable result as both tools in their standard configuration calculate maps which are unsimilar to the real track. However only the adapted Gmapping approach achieved to calculate an accurate grid map of the whole race track. The maximum local error of the Cartographer map in some sections made its use unfeasible for our use case. The autonomous time attack challenge in Berlin was therefore successfully performed with the offline calculated map of Gmapping. The map served as input for the planning of the global racing trajectory.

Nonetheless, advantages and disadvantages of either tool can be found. The drawback of Gmapping resides in the computational restrictions. The application to a real-time SLAM could not be realized in this work. The advantage of the Gmapping approach in our case resides in the accuracy over the entire track. The Gmapping map achieves a better compatibility to our LIDAR localization functionality. The experimental localization results were more reliable with Gmapping maps than with Cartographer maps for different scenarios. We assume this is due to the fact that the implemented LIDAR localization uses the same scan representation as Gmapping as an input while the Cartographer processes the data in the

raw point cloud data format. The different characteristics of these representations could lead to a worse scan matching result.

The adapted Cartographer approach did not calculate an accurate global map. We cannot reach a definite conclusion whether or not it could be adapted further to calculate a correct map for the use case. One drawback of this approach lies in the large amount of tuning parameters and their complex interdependence. This makes the adaption to a specific environment a difficult task. Contrariwise, the Cartographer possesses a greater potential in the field of large scale SLAM. In the current work we created the maps offline. In the future, the Cartographer can possibly be applied for real-time SLAM with local path-planning. Furthermore, a localization function is implemented in Cartographer which could overcome the drawback in localization mode. In a use case with a 3D laser scanner the Cartographer can possibly achieve better scan matching results while Gmapping would not be able to extract additional information of the 3D layers. The errors in the map, we could not prevent for the quasi 2D case, can possibly be reduced with an full 3D sensor setup as the scan matching can rely on a bigger number of features.

CONTRIBUTIONS

Felix Nobis as a first author initiated the idea of this paper and contributed essentially to its content. Johannes Betz contributed to the conception and realization of this research and the structure of this paper. Leonhard Hermansdorfer contributed to software components supporting this research. Markus Lienkamp made an essential contribution to the conception of the research project. He revised the paper critically for important intellectual content. He gave final approval of the version to be published and agrees to all aspects of the work. As a guarantor, he accepts the responsibility for the overall integrity of the paper.

ACKNOWLEDGMENT

We thank the Roborace team for giving us the opportunity to work with them and using their vehicles for our research. We express gratitude to Continental Engineering Service for funding for the underlying research project. We would like to thank the Bayrische Forschungsförderung for funding for the research project rAIcing. Further, we like to thank Luis Paarup whose master thesis contributed to this research.

REFERENCES

- [1] Yassin Abdelrasoul, Abu Bakar Sayuti HM Saman, and Patrick Sebastian. 2016. A quantitative study of tuning ROS gmapping parameters and their effect on performing indoor 2D SLAM. In *2016 2nd IEEE International Symposium on Robotics and Manufacturing Automation - IEEE-ROMA 2016*. IEEE, Piscataway, NJ, 1–6. <https://doi.org/10.1109/ROMA.2016.7847825>
- [2] Autonomoustuff. 2018. Ibeo ScaLa B2. <https://autonomoustuff.com/product/ibeo-scala/>
- [3] Johannes Betz, Alexander Wischniewski, Alexander Heilmeyer, Felix Nobis, Tim Stahl, Leonhard Hermansdorfer, Boris Lohmann, and Markus Lienkamp. 2018. What can we learn from autonomous level 5 motorsport? *chassis.tech plus | 9. Int. Münchner Fahrwerk-Symposium* (2018).
- [4] Michael Bosse and Robert Zlot. 2008. Map Matching and Data Association for Large-Scale Two-dimensional Laser Scan-based SLAM. *The International Journal of Robotics Research* 27, 6 (2008), 667–691. <https://doi.org/10.1177/0278364908091366>
- [5] Guillaume Bresson, Zayed Alsayed, Li Yu, and Sebastien Glaser. 2017. Simultaneous Localization and Mapping: A Survey of Current Trends in Autonomous Driving. *IEEE Transactions on Intelligent Vehicles* 2, 3 (2017), 194–220. <https://doi.org/10.1109/TIV.2017.2749181>
- [6] Katie Burke. 2018. The Fast and the Driverless: Munich Team Takes Home Roborace Victory: Researchers at Technical University of Munich turn to NVIDIA DRIVE to fill their need for speed at Berlin event. <https://blogs.nvidia.com/blog/2018/06/29/fast-and-driverless-munich-robtorace-victory/>
- [7] Felix Endres, Jurgen Hess, Jurgen Sturm, Daniel Cremers, and Wolfram Burgard. 2014. 3D Mapping With an RGB-D Camera. *IEEE Transactions on Robotics* 30, 1 (2014), 177–187. <https://doi.org/10.1109/TRO.2013.2279412>
- [8] Google. 2018. Cartographer. <https://google-cartographer-ros.readthedocs.io/en/latest/>
- [9] Giorgio Grisetti, Cyrill Stachniss, and Wolfram Burgard. 2007. Improved Techniques for Grid Mapping With Rao-Blackwellized Particle Filters. *IEEE Transactions on Robotics* 23, 1 (2007), 34–46. <https://doi.org/10.1109/TRO.2006.889486>
- [10] Fredrik Gustafsson. 2010. Particle filter theory and practice with positioning applications. *IEEE Aerospace and Electronic Systems Magazine* 25, 7 (2010), 53–82. <https://doi.org/10.1109/MAES.2010.5546308>
- [11] Wolfgang Hess, Damon Kohler, Holger Rapp, and Daniel Andor. 2016. Real-time loop closure in 2D LIDAR SLAM. In *2016 IEEE International Conference on Robotics and Automation, Stockholm, Sweden, May 16th–21st*, Allison Okamura and Arianna Menciassi (Eds.). IEEE, Piscataway, NJ, 1271–1278. <https://doi.org/10.1109/ICRA.2016.7487258>
- [12] Kamarulzaman Kamarudin, Syed Muhammad Mamduh, Ali Yeon Md Shakaff, and Ammar Zakaria. 2014. Performance analysis of the Microsoft Kinect sensor for 2D Simultaneous Localization and Mapping (SLAM) techniques. *Sensors (Basel, Switzerland)* 14, 12 (2014), 23365–23387. <https://doi.org/10.3390/s141223365>
- [13] Stefan Kohlbrecher, Oskar von Stryk, Johannes Meyer, and Uwe Klingauf. 2011. A flexible and scalable SLAM system with full 3D motion estimation. In *2011 IEEE International Symposium on Safety, Security, and Rescue Robotics*. IEEE, 155–160. <https://doi.org/10.1109/SSRR.2011.6106777>
- [14] M. Montemerlo, N. Roy, and S. Thrun. 2003. Perspectives on standardization in mobile robot programming: The Carnegie Mellon navigation (carmen) toolkit. In *Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003)* (Cat. No.03CH37453). IEEE, 2436–2441. <https://doi.org/10.1109/IROS.2003.1249235>
- [15] Michael Montemerlo, Sebastian Thrun, Daphne Koller, and Ben Wegbreit. 2003. FastSLAM 2.0: An Improved Particle Filtering Algorithm for Simultaneous Localization and Mapping that Provably Converges. *Proceedings of the Sixteenth International Joint Conference on Artificial Intelligence (IJCAI), Acapulco, Mexico (2003)* (2003).
- [16] Hans P. Moravec. 1988. Sensor Fusion in Certainty Grids for Mobile Robots. *AI Magazine* 9, 2 (1988), 61–74.
- [17] Raul Mur-Artal and Juan D. Tardos. 2017. ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras. *IEEE Transactions on Robotics* 33, 5 (2017), 1255–1262. <https://doi.org/10.1109/TRO.2017.2705103>
- [18] A. Nüchter, M. Bleier, J. Schauer, and P. Janotta. 2017. IMPROVING GOOGLE'S CARTOGRAPHER 3D MAPPING BY CONTINUOUS-TIME SLAM. *ISPRS - International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences XLII-2/W3* (2017), 543–549. <https://doi.org/10.5194/isprs-archives-XLII-2-W3-543-2017>
- [19] NVIDIA. 2018. NVIDIA DRIVE PX2. <https://www.nvidia.de/self-driving-cars/drive-platform/>
- [20] E. B. Olson. 2009. Real-time correlative scan matching. In *2009 IEEE International Conference on Robotics and Automation*. IEEE, 4387–4393. <https://doi.org/10.1109/ROBOT.2009.5152375>
- [21] OXTS. 2018. OXTS RT4000. <https://www.oxts.com/products/rt4000/>
- [22] Scott Pendleton, Hans Andersen, Xinxi Du, Xiaotong Shen, Malika Meghiani, You Eng, Daniela Rus, and Marcelo Ang. 2017. Perception, Planning, Control, and Coordination for Autonomous Vehicles. *Machines* 5, 1 (2017), 6. <https://doi.org/10.3390/machines5010006>
- [23] ROS. 2018. ROS. <http://www.ros.org/>
- [24] Joao Machado Santos, David Portugal, and Rui P. Rocha. 2013. An evaluation of 2D SLAM techniques available in Robot Operating System. In *2013 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*. IEEE, 1–6. <https://doi.org/10.1109/SSRR.2013.6719348>
- [25] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. 2006. *Probabilistic Robotics*. MIT Press, Cambridge, Massachusetts and London, England.